Fidelity-Graded Boundaries and Syntactic Confinement in OSTADIX

Research and engineering note, 11 November 2026. Maintainer/publication identity: Lee Daghlar Ostadi. Prepared with AI assistance; see development disclosure. Inspected source baseline: `1da570dcc61ef10285fc8c8b78a20ca087081daf`, with accompanying public-contract changes. This note preserves the separate, dated technical whitepaper.

Abstract. A polyglot expression boundary has at least three contracts: the representation that crosses it, the distinctions the receiver preserves, and the grammar into which the receiver inserts it. An exact wire round trip establishes only the first. OSTADIX provides concrete counterexamples in which CBOR preserves a value while a JavaScript crossing changes Python type dispatch or integer parity. A separate witness family concerns data that becomes receiver syntax. We state the existing definite/possible-loss abstraction as a Galois insertion over finite nonempty families of structural observations, prove its sequential-composition law, and delimit the failure of severity merging to model arbitrary execution alternatives. We then give a conditional source-confinement proposition: admitted literal holes preserve the surrounding grammar skeleton if their renderer, lexical-context check and guest grammar satisfy explicit assumptions. This proposition is an obligation for implementation, not a proof of the actual guest lexers. The note connects those obligations to source locations, profile boundaries, regression oracles and prior work, without claiming the first polyglot system or a universal semantic-preservation theorem.

1 The boundary, not the transport, is the unit of the claim

An O expression `LANG^(body)_LANG` names an evaluator in the syntax. Its parser node records language, environment identity, attributes and children. An evaluator tag is not a static theorem about the foreign expression's result type. In particular, a Python region can return different OValue families or perform effects. O-core's statically checked machine language is a separate layer.

For a value crossing from language A to language B, distinguish lifting, transport, receiver projection and execution. Lifting maps an observed native result into OValue. Framed CBOR carries that OValue. A receiver may bind it as an object, encode a literal, deliberately present it, or insert code. The guest then parses and evaluates the resulting input. A law for one arrow does not establish a law for their composition.

This distinction fits the operational-boundary perspective of Matthews and Findler: their lump and natural embeddings give different accounts of values moving between languages, rather than treating interoperability as a serialization detail [1]. Our implementation question is narrower: which distinctions can each actual adapter preserve, and what evidence supports that answer? OValue native capsules are an explicit representation option, but their existence does not make every adapter a natural embedding or establish the paper's semantic theorems for OSTADIX.

A useful record therefore names the producer profile, value representation, receiving adapter, source context, observation and claim scope. An engine executable's presence is neither permission to execute it nor proof of its semantics. Likewise, a persistent proxy is not necessarily a persistent guest: the JavaScript native bridge spawns a fresh Node.js process per execution.

The concrete implementation coordinates are `parser.rs`, `value.rs`, `wire.rs`, `backend_catalog.inc.rs`, `backend.rs` and `eval_core.rs` in `crates/ostadix-api/src`. The current public contract links them and separates ordinary OIR admission from the source/intent gate, initial scope and project execution. This note studies boundary judgments within that separation. It does not close those larger authority or reproducibility obligations.

2 Witness family A: encoded values survive, decisions change

The existing experiments in `benchmarks/fidelity` run Python to JavaScript to Python through real O execution plans. They retain ordinary-scalar and tagged-OValue CBOR controls, graph and serial results, runtime crossing observations and a known-value HGraph analysis. Their recorded environment and source date belong to the September 2026 result, not a newly asserted November execution.

Witness	Original and returned values	Observable downstream distinction
`type_tag.0`	binary64 `1.0` becomes integer `1`	Python type dispatch selects a different branch although numeric equality holds.
`integer_precision.0`	`9007199254740993` becomes `9007199254740992`	The oddness test changes from true to false.

Both CBOR controls preserve the original values exactly. The binary64 payload is `fb3ff0000000000000`; the integer payload is `1b0020000000000001`, excluding the transport's length prefix. These controls localize the observed loss beyond the wire encoding. They do not show that every use of CBOR, or every consumer of the same payload, is correct.

The JavaScript stdout path explains the witnesses. Its numeric projection uses JavaScript Number, whose binary64 representation cannot distinguish the adjacent large integers in this case. The textual output `1` is then lifted as an integer. A carrier capable of representing exact integers does not compel this adapter to use an exact-integer mapping. Nor does a numerically unchanged result preserve a producer's type tag.

The runtime morphism profile and capability analysis also differ. The float witness lies within the narrow JavaScript runtime profile and is assessed structurally. The large integer lies outside that profile and produces `IntegerOutOfRange`; a known-value capability graph separately reports structural precision loss. Treating that rejection as a successful morphism proof would erase a material qualification.

The graph analysis uses the actual observed intermediate value. It does not evaluate arbitrary Python source statically. Its locally lossless return leg must retain loss accumulated on the outgoing leg. The exact observation and accumulation assertions are in `tests/fidelity_evaluation.rs`; commands, tools, result identities and nonclaims are in the witness guide and dated results.

This family refutes transport success as a sufficient semantic criterion. It is not a proof that every boundary loses information, an inter-language performance comparison or a census of backend correctness. Choosing a different exact binding could change the outcome; that is a different adapter contract to implement and test.

3 Structural fidelity as an information interval

Let U be the vocabulary of annotation kinds. It is not globally finite: `BackendSpecific` admits string labels. A concrete structural observation is a finite loss set L. Lossless is the empty set. Write C0 for the finite subsets of U, and let C contain the finite, nonempty families S of members of C0, ordered by inclusion. An abstract assessment is an interval [D,P] of finite sets satisfying D subset P: losses in D are definite, and losses outside P are excluded.

The information order puts a narrower interval below a wider one:

$$[D_1, P_1] \sqsubseteq [D_2, P_2] \quad \iff \quad D_2 \subseteq D_1 \ \wedge \ P_1 \subseteq P_2.$$

Define abstraction and concretization:

$$\alpha(S) = [\bigcap S, \bigcup S], \qquad \gamma([D, P]) = \{L \mid D \subseteq L \subseteq P\}.$$

Both maps are well defined. A finite nonempty S has finite union and intersection. For finite P, its interval contains finitely many sets and at least D. No globally finite universe is required. The adjunction is:

$$\alpha(S) \sqsubseteq [D, P] \quad \iff \quad S \subseteq \gamma([D, P]).$$

Proof. The left side says D is contained in every member of S and every member is contained in P. That is exactly membership in the right-side interval for every member. Further, D and P both belong to gamma([D,P]); their intersection is D and their union is P. Consequently alpha(gamma(a)) equals a. The connection is a Galois insertion on these stated posets.

This is an application of the abstraction/concretization discipline, not a new general abstract-interpretation foundation [9]. In particular, these domains are not claimed to be complete lattices. An empty family would require a representable bottom, and arbitrary infinite families may require an infinite possible-loss bound. The implementation's invalid directly constructed bounds do not supply that bottom.

Monotonicity follows immediately: S subset T makes the intersection smaller and union larger, hence alpha(S) below alpha(T); interval inclusion makes gamma monotone. For a concrete loss L, the point embedding is [L,L], giving the testable equivalence `from_concrete(c).precision_leq(a) == a.concretization_contains(c)` for structural observations.

The full algebra note provides the maintained proof and executable-check coordinates. These are theorems about loss judgments. Their interpretation as truthful backend judgments additionally requires an oracle or semantic argument for each transfer rule. A sound interval calculus cannot repair an unsound premise supplied by an adapter.

4 Composition, alternatives and convergence are different orders

For successive structural crossings, concrete cumulative loss is set union. The implemented sequential operator computes:

$$[D_1, P_1] \text{ then } [D_2, P_2] = [D_1 \cup D_2, P_1 \cup P_2].$$

Soundness. If each Li belongs to gamma(ai), union of the lower inclusions and union of the upper inclusions place L1 union L2 inside the resulting interval. The same inclusions show monotonicity of `then` in either operand under the information order. The claim assumes that losses accumulate as recorded; it is not a theorem that a later program cannot reconstruct a particular distinction from independent information.

For alternative structural outcomes, interval hull is different:

$$[D_1, P_1] \text{ join } [D_2, P_2] = [D_1 \cap D_2, P_1 \cup P_2].$$

It is the least information-order upper bound. It may introduce extra alternatives because the interval forgets correlations. For example, alternatives losing only A or only B produce an interval from the empty set to {A,B}, also admitting neither and both. This overapproximation is explicit.

`NativeCapsule` and `Unsupported` are isolated sentinel outcomes in the current enum. Its severity merge returns `Unsupported` for `Lossless.join_paths(Unsupported)`. Yet the concretization of that sentinel omits the lossless outcome. Therefore severity precedence is not an upper bound for that mixed family. There is no theorem that it models every `any` or `race` result. A success-selecting execution must retain alternatives separately or bind the assessment to evidence of its actual selection; it must not silently substitute a severity maximum for those semantics.

Termination is a third question. Over a fixed finite vocabulary B, structural intervals have 3 to the power |B| states. The solver's accumulative update grows definite and possible coordinates together, whereas the information order reverses the definite coordinate. Increasing definite losses can refine information, while increasing possible losses broadens it. A Galois insertion alone is not a convergence proof for that update order.

Fresh unbounded backend labels invalidate a finite-height argument unless the transfer vocabulary is bounded or a widening is supplied. Convergence budgets must fail closed rather than declare success after arbitrary truncation. Existing generated-set checks, finite three-kind enumeration and sentinel regressions exercise these algebraic boundaries in `value.rs`. They are bounded executable checks plus a mathematical argument; they do not certify all solver transfer rules or turn an execution receipt into admission authority.

5 Conditional syntactic confinement for literal holes

Semantic value preservation does not imply grammatical confinement. A text value may arrive exactly and still become an operator, delimiter, command substitution or additional statement when parsed by its receiver. Su and Wassermann formalize injection through changes to the intended syntactic structure and analyze a grammar-based prevention algorithm [2]. Our proposition uses an explicit skeleton obligation; it is not a claim to implement their complete algorithm.

Fix a guest-language profile p with a declared grammar G, a supported literal-value domain V, a partial renderer r, and admitted expression-hole contexts C. A skeleton replaces each designated literal subtree with its hole identifier while retaining all surrounding syntax. For an ordinary value hole, require the following premises.

1. **Literal law.** For every admitted v in V, r(v) parses as one permitted literal subtree (or an explicitly allowed literal-construction subtree). Its evaluation has the declared projection of v; any loss is reported separately.

2. **Context law.** For every admitted context C and tuple of renderings, parsing the filled context succeeds and its skeleton equals the checked template skeleton. The hole cannot terminate a guest string/comment, merge with a neighboring token or occupy an unapproved grammar position.

3. **Closure law.** Nested O results and variable holes use that same checked path. No hidden renderer or implicit fallback subsequently reinterprets their output. Unsupported values and profiles reject before receiving-backend execution. Computing a child value may already have run producer effects; this is not a whole-document rollback law.

4. **Runtime agreement.** The actual guest grammar, dialect, encoding and relevant lexical options match the checked profile. Parent code performs no later `eval`, shell expansion or other reinterpretation of the resulting value unless that next boundary is separately modeled.

Proposition. Under these premises, changing admitted ordinary-hole values preserves the surrounding checked grammar skeleton. The holes contribute only their declared literal subtrees.

Proof. For one hole this is the context law with the literal law supplying an admitted subtree. For finitely many holes, apply the context law to the full tuple, or induct on designated holes when the profile establishes compositional context closure. Nested expression evaluation supplies a member of V and then uses the same renderer by the closure law. Runtime agreement transfers the checked skeleton statement to the parser actually used. No claim follows for an explicit code hole or a value outside V.

Bash supplies a concrete reason for the runtime-agreement premise: a quoted string used as an operand of `[[ ... -gt ... ]]` can undergo arithmetic evaluation, including array-subscript command substitution. The bounded guard therefore refuses ordinary holes in that nested context. Authored `eval`, `let`, `declare -i`, or pre-existing integer-variable attributes remain semantic reparsing sinks, as does Python `eval`. Literal ingress alone does not make such a parent safe or remove its authority. The proposition is conditional on the absence of unmodeled subsequent reinterpretation; the lexical checks do not prove that premise for arbitrary guest code.

The context law is substantial: it cannot be replaced by checking only that a hole is outside quotes. Receiver tokenization, Unicode, prefixes, escape forms, comments and grammar positions matter. The proposition does not prove OSTADIX's actual scanner satisfies these premises. Lexical guards and literal encoders support a bounded implementation profile; comprehensive guest parsers, dialect modeling and adversarial regression remain distinct obligations. No sandbox, authorization, data confidentiality or absence of dangerous parent operations follows from syntactic confinement alone.

6 Witness family B: program data becomes receiver grammar

This family holds transport constant and varies the receiver source context. The seven original inputs were run against the frozen baseline binary (SHA-256 `b318ec235bf1f86f81c857d6c6836f51657176a403b339cc8925a7c210791bbd`) and archived baseline backends, using both serial and graph execution under a five-second bound. The retained baseline records and sources distinguish controls, failures and injected observations. These are baseline observations, not revised-source successes.

Original witness	Observed baseline result on both paths	Revised literal-contract obligation
`python_apostrophe`	`[7, "O'Brien"]`	Preserve the unquoted value hole.
`sql_quoted_control`	Text `OBrien`	Reject the quoted O hole; use unquoted SQL literal insertion.
`sql_quoted_apostrophe`	External five-second timeout	Reject before dispatch; transport EOF handling is a separate fix.
`sql_union_injection`	Added row `spliced-as-code`	Reject the quoted O hole; no injected row on the migrated literal form.
`bash_quoted_injection`	Output includes `SPLICED-AS-CODE` as an extra command result	Reject the quoted O hole; keep literal payload from becoming shell grammar.
`bash_native_braces`	Original payload printed literally	Retain native `${s}` binding expansion semantics.
`python_quoted_hole`	Error with Python syntax failure	Reject the invalid O hole context before executing Python.

A timeout establishes only that the external bound expired; it is not a successful parse, a demonstrated infinite loop or proof of the cause. The injected SQL row and shell marker are observations of receiver-grammar interpretation, not claims of a compromised external system. All inputs use local demonstration values.

For example, an O binding used in Python should have the value form `python^(__oval_result__ = $name)_python`. Manually adding guest quotes around an already literal-rendered hole changes the contract. The SQL form is `sql^(SELECT $name;)_sql`, with no caller-supplied string quotes. In a shell body, native `${name}` remains guest syntax referring to a supplied binding; it is not the O hole `$name`. That distinction must be checked by the actual parser, not inferred from a documentation convention.

An explicit code marker such as `$!name` has a different intent. Its plan-visible `code($name)` operation identifies a request to use content as receiver code. Only an immediate code-call hole selects this projection; storing its returned ordinary value does not carry a hidden code permission. It is not an authority grant, an escaping theorem or a new universal wire value. Its projection is conservatively identified as reinterpretation. It is permitted even for an otherwise unknown executable profile, with no confinement claim or extra authority. A later ordinary hole after dynamic code in the same body is rejected when its context cannot be established.

The distinction must survive recursive nesting and generated programs. Classifying a container without inspecting its members can hide a code-like or opaque member. A fallback that silently turns an unsupported value into executable text violates the closure premise even if the common scalar cases are safe.

These tests must compare complete typed results, source diagnostics and absence of backend effects on refusal. A parse success alone is insufficient: the exact rendered guest body, guest version/dialect and downstream observation are necessary. Rejecting a case is valid bounded behavior; accepting it with altered semantics is not rescued by a transport-success flag. Conversely, intentional HTML presentation must not be advertised as executable-language confinement or browser sanitization.

7 Implementation obligations and an auditable evaluation

The declarative backend catalog selects adapter ownership, renderer, execution mode and capabilities. The ordinary parser lowers executable structure into OIR and a validated ExecutionPlan. Profile-specific literal/context logic must therefore be checked at both source parsing and lowered-OIR validation: public consumers can supply already-lowered programs. Otherwise, a correct parser restriction can be bypassed by a forged or reconstructed program.

The accompanying source contract separates known literal/lexical profiles from presentation and unsupported profiles. Conservative exclusions include Python f/t-strings, JavaScript/Nix interpolation, complex shell substitutions/parameter expansions/array subscripts, heredocs, ambiguous JavaScript slash, Nix indented strings/path prefixes, and bare Python/JavaScript backslashes. An unsupported prefix can leave later holes unavailable even when the construct appears closed; simple native shell `${name}` and guest code without O holes are separate supported cases. The checked-in seven witness fixtures and contract test target preserve the inputs and revised oracles; a source file is not itself an executed result. Python, SQL, Bash/shell, JavaScript and the declared Nix family require their actual profile-specific rules. HTML, text, Markdown and LaTeX are intentional presentation surfaces. Unknown executable profiles cannot inherit a confinement claim merely by sharing a generic renderer. TypeScript is not a registered backend in the inspected catalog and is not inferred from JavaScript support. The authoritative final profile set and explicit-code restrictions belong in the SPEC, with source and regression coordinates.

The Rust trace now records actual prepared-fragment observations, not only an on-demand classification API. `source_renderings` names parent/child IDs, language/renderer, value/code role and fidelity, and hashes the canonical value and rendered fragment without disclosing their raw contents. Phase `prepared` is not guest execution. Optional `final_body_sha256` covers only the completed receiver splice buffer, before backend-owned wrappers, preludes or control framing. Partial preparation and guest failure retain their actual observations; denied/unused quotes invent none. Trusted inline presentation-only worker fast paths are outside coverage. The additive trace fields neither change admission/wire versions nor make a post-runtime body pre-admitted. The Python reference and C17 now have separate bounded data/code guards and real-engine tests. This does not transfer Rust admission, trace, deadline or checkpoint guarantees to those editions; the legacy C++ prototype remains outside this qualification.

Evaluation should retain two independent suites. Family A preserves exact transport controls and observes changed type/parity through original engines. Family B observes source reinterpretation or validates rejection/literal behavior in each declared context. Include adversarial quotes, backslashes, line endings, comments, Unicode, token adjacency, nested O values, recursive collections, ordinary bindings, explicit code and unsupported values. Run graph and serial paths against the same admitted program and compare results without normalizing away a mismatch.

Every result should identify source commit and patch, compiler/executable digests, guest versions, platform, command, input and rendered-source identities, stdout/stderr, exit/timeout, typed value, relevant profile, rejected cases and negative controls. Missing runtimes are unmet prerequisites, not a passing zero-case suite. A timeout or infrastructure failure is not a semantic result. The finite step timeout is also separate from heartbeat and total-operation deadlines.

The ordinary V6 admission boundary binds lowered OIR, not universally every original source byte or caller initial-scope value. The explicit source/intent front door adds its own check before fresh admission. `ProjectCoordinator` and Request scheduling retain separate authorities. These distinctions matter when constructing a reproducibility package: retain what actually ran rather than infer a complete experiment identity from one graph digest.

No new performance improvement is claimed by this note. Protocol framing, exact numeric binding, lexical validation and extra checks have costs that need matched measurements. Component lowering, object construction and successful emulated boots likewise do not establish a whole-system efficiency result,

physical device support or general original-runtime lowering.

8 Position among prior work and scope of the contribution

Several lines of prior work address different parts of this problem. Their existence makes a broad claim that OSTADIX first enables fine-grained polyglot composition untenable without a much more specific comparison.

Matthews and Findler provide operational models for multi-language boundaries, including lump and natural embeddings [1]. Su and Wassermann characterize injection by unintended syntactic structure [2]. Bravenboer, Dolstra and Visser use syntax embeddings to address injection independently of a particular host/guest pairing [3]. These works motivate explicit boundary semantics; this note does not inherit their correctness results merely by using similar terminology.

Omar and colleagues' **Safely Composable Type-Specific Languages** is ECOOP 2014 and includes six authors, not only Omar and Aldrich [4]. It associates literal parsing/elaboration with expected types. Omar and Aldrich's **Reasonably Programmable Literal Notation** is PACMPL 2, ICFP 2018, article 106; it develops typed literal macros and reasoning principles [5]. Those are verified bibliographic identities. Evaluator tags are not equivalent to their type-directed elaboration discipline, and OSTADIX's source-level guest guards do not establish the same hygiene or typing theorems.

Van Wyk and Schwerdfeger's GPCE 2007 context-aware scanning passes parser-state information to lexical disambiguation [6]. Eco, by Diekmann and Tratt, is an SLE 2014 language-composition editor using language boxes [7]. PyHyp's PHP/Python study is the ECOOP 2016 paper by Barrett, Bolz, Diekmann and Tratt [8]. Their respective parsing, editing and runtime composition results are distinct; a single citation should not collapse them into one 2016 system or imply that language boxes solve all value fidelity.

GraalVM's official polyglot documentation describes an interoperability message protocol between supported Truffle languages and host/guest value exchange [10]. That is an important comparison for value boundaries, not evidence that either system universally preserves all foreign distinctions or supports arbitrary engines without implementation work.

The useful contribution proposed here is a precise, inspectable contract: retain evaluator choice in syntax, grade value losses independently, distinguish literal/presentation/code projections, and bind concrete observations to their source and profiles. The structural interval proofs are established on their stated domain. The confinement proposition is conditional. Actual guest-lexer adequacy, backend loss rules, complete profile execution, native runtime lowering and cross-ISA semantic identity remain separately evidenced work.

This document was composed with AI assistance under maintainer direction. Its publication identity does not assert that the maintainer personally typed every implementation or supplied every proof step. Claims should be judged by their arguments, source and retained experiments, with unresolved assumptions preserved.

References and verification

Primary author, institution, conference and project sources verified on 11 November 2026. The Markdown edition and citation-verification JSON retain the source URLs and claim mapping. No cited paper's theorem or experiment is transferred to OSTADIX.

References

[1] Jacob Matthews, Robert Bruce Findler. *Operational Semantics for Multi-Language Programs*. POPL, 2007. https://users.cs.northwestern.edu/~robby/pubs/papers/popl2007-mf-bw-save.pdf.

[2] Zhendong Su, Gary Wassermann. *The Essence of Command Injection Attacks in Web Applications*. POPL, 2006. https://www.cs.ucdavis.edu/~su/publications/popl06.pdf.

[3] Martin Bravenboer, Eelco Dolstra, Eelco Visser. *Preventing Injection Attacks with Syntax Embeddings*. GPCE; corresponding TUD-SERG-2007-003 report, 2007. https://repository.tudelft.nl/file/File_a8f8bddf-3f95-4790-83fb-f6d0c107fe32.

[4] Cyrus Omar, Darya Kurilova, Ligia Nistor, Benjamin Chung, Alex Potanin, Jonathan Aldrich. *Safely Composable Type-Specific Languages*. ECOOP, 2014. https://potanin.github.io/files/OmarKurilovaNistorChungPotaninAldrichECOOP2014.pdf.

[5] Cyrus Omar, Jonathan Aldrich. *Reasonably Programmable Literal Notation*. PACMPL 2 (ICFP), article 106, 2018. https://www.cs.cmu.edu/~aldrich/papers/icfp18-notation.pdf.

[6] Eric R. Van Wyk, August C. Schwerdfeger. *Context-Aware Scanning for Parsing Extensible Languages*. GPCE, 2007. https://www-users.cse.umn.edu/~evw/pubs/vanwyk07gpce/vanwyk07gpce.pdf.

[7] Lukas Diekmann, Laurence Tratt. *Eco: A Language Composition Editor*. SLE, 2014. https://soft-dev.org/pubs/html/diekmann_tratt__eco_a_language_composition_editor/.

[8] Edd Barrett, Carl Friedrich Bolz, Lukas Diekmann, Laurence Tratt. *Fine-grained Language Composition: A Case Study*. ECOOP, LIPIcs 56, 3:1-3:27, 2016. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2016.3.

[9] Patrick Cousot, Radhia Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL, 1977. https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml.

[10] GraalVM Project. *Polyglot Programming*. Official reference manual, accessed 11 November 2026. https://www.graalvm.org/latest/reference-manual/polyglot-programming/.

Rebuild: pdflatex fidelity-boundaries.tex twice. The companion BibTeX file retains reusable structured bibliographic entries. Source baseline and development disclosure are stated on the first page.